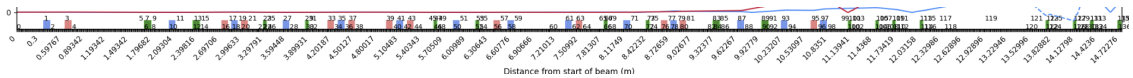
beamline_optimization

April 4, 2026

1 Beamline Optimization - Initialization

This notebook performs sequential optimization of the UH beamline using custom modules and beam parameters.

Help for associated beamline element indexing: image.png



[1]:
```
import os, sys
from pprint import pprint
# pprint(sys.path)

current_dir = os.path.abspath(os.getcwd)

project_root = os.path.abspath(os.path.join(current_dir, '..'))

if project_root not in sys.path:
    sys.path.insert(0, project_root)
pprint(sys.path)
# from backend.beamline import *
from beamline import *
```

```
['C:\\Users\\yi_lu\\my_files\\research\\FELsim\\backend',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\python311.zip',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\DLLs',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\Lib',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim',
 '',
 'C:\\Users\\yi_lu\\AppData\\Roaming\\Python\\Python311\\site-packages',
 'C:\\Users\\yi_lu\\programming_lang\\Anaconda\\envs\\felsim\\Lib\\site-
packages']
```

[2]:
```
# Import Python modules
%matplotlib ipympl
import sys
import time
from pathlib import Path
```

```python
import numpy as np
import pandas as pd
import sympy as sp
import sympy.plotting as plot

# Import Beam Dynamics modules
from ebeam import beam
from beamline import lattice, Beamline
from schematic import draw_beamline
from excelElements import ExcelElements
from beamOptimizer import beamOptimizer
from AlgebraicOptimization import AlgebraicOpti
```

[3]:
```python
# Initial Beam Parameters
Energy = 40  # MeV
f = 2856e6  # Hz
bunch_spread = 2  # ps
energy_std_percent = 0.5  # 0.3 % Energy spread from M. Hadmack Rev. Sci.␣
↪Instrum. 84, 063302 (2013); doi: 10.1063/1.4809938
h = 5e9  # 1/s

epsilon_n = 8  # pi.mm.mrad
x_std = 0.8  # mm
y_std = 0.8  # mm
nb_particles = 1000

relat = lattice(1,fringeType=None)
relat.setE(E=Energy)
norm = relat.gamma * relat.beta
epsilon = epsilon_n / norm
print(relat.gamma)
print(relat.beta)
print(epsilon)
x_prime_std = epsilon / x_std
y_prime_std = epsilon / y_std

tof_std = bunch_spread * 1e-9 * f
gen_tof = np.random.normal(0, tof_std, size=(nb_particles, 1))
energy_std = energy_std_percent * 10

ebeam = beam()
beam_dist = ebeam.gen_6d_gaussian(0,␣
↪[x_std,x_prime_std,y_std,y_prime_std,tof_std,energy_std], nb_particles)
tof_dist = beam_dist[:,4] / f
beam_dist[:,5] += h * tof_dist
```

```
# MkV FEL Undulator matching
K = 1.2
lambda_u = 2.3  # Undulator period in cm

beta_ym = relat.gamma / (K * (2 * np.pi / (lambda_u * 1e-2)))
print(beta_ym)
y_std_m = np.sqrt(epsilon * 1e-6 * beta_ym)
print(y_std_m)
y_prime_std_m = np.sqrt(epsilon * 1e-6 / beta_ym) # Beam at waist so Gamma(z_w)␣
↪= 1 / Beta(z_w)
print(y_prime_std_m)
```

```
79.2794183839214
0.9999204452035254
0.10091694227956337
0.2418394669906551
0.00015622323620773033
0.0006459790792284825
```

[4]:
```
# Load Beamline from Excel
current_path = Path.cwd()


file_path = current_path.parents[1] / 'beam_excel' / 'Beamline_elements.xlsx'



excel = ExcelElements(file_path)
df = excel.get_dataframe()
beamlineUH = excel.create_beamline()
schem = draw_beamline()
line_UH = relat.changeBeamType("electron", Energy, beamlineUH)

# Optimizer and beamline truncation
print('Number of elements in beamline: ' + str(len(line_UH)))
segments = 138
line = line_UH[:segments]
opti = beamOptimizer(line, beam_dist)
```

```
Number of elements in beamline: 138
```

[5]:
```
# Optimization - First Quadrupole Doublet

variables = {
    1: ["I", "current", lambda num:num],
    3: ["I2", "current", lambda num:num],
}
startPoint = {
```

```
    "I": {"bounds": (0,10), "start": 1},
    "I2": {"bounds": (0, 10), "start": 1},
}

objectives = {8: [{"measure": ["x", "alpha"], "goal": 0, "weight":1},
                  {"measure": ["x", "beta"], "goal": 0.1, "weight":0.0}],
              9: [{"measure": ["y", "alpha"], "goal": 0, "weight":1},
                  {"measure": ["y", "beta"], "goal": 0.1, "weight":0.5}]}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)

# old values
# line[1].current = 0.9989681933
# line[3].current = 1.044851479
```

```
x variables:
indice 1 new current value: 0.8423770026220159
indice 3 new current value: 1.05546811783588

y objectives:
indice 8: x alpha value of -0.00064138684
indice 8: x beta value of 0.9966604
indice 9: y alpha value of -0.00031623873
indice 9: y beta value of 0.13155359
Final difference: 0.00012458147830329835

Total time: 97.55513230001088 s
Total iterations: 64
```

[6]:
```
# Optimization - First Chromacity Quad

variables = {10: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {15: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 10 new current value: 4.810156250000008

y objectives:
indice 15: x disper value of 4.454928e-07
Final difference: 1.5551601144554207e-14
```

```
Total time: 74.5879169999971 s
Total iterations: 40
```

[7]:
```
# Optimization - Quadrupole Triplet

truncated_line = line[:17]
# Add half a quad
#truncated_line.append(qpfLattice(current = 3.816604, length=0.0889/2))

variables = {
    16: ["I", "current", lambda num:num],
    18: ["I2", "current", lambda num:num],
    20: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 5},
    "I3": {"bounds": (0, 10), "start": 3},
}

objectives = {25: [{"measure": ["x", "alpha"], "goal": 0, "weight":1},
                   {"measure": ["x", "beta"], "goal": 0.1, "weight":0.5}],
               26: [{"measure": ["y", "alpha"], "goal": 0, "weight":1},
                   {"measure": ["y", "beta"], "goal": 0.1, "weight":0.5}]}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)

# old values
# line[16].current = 2.4
# line[18].current = 5.108214683
# line[20].current = 3.142089844
```

```
x variables:
indice 16 new current value: 2.6484345842598627
indice 18 new current value: 5.021757024275974
indice 20 new current value: 3.131977746801887

y objectives:
indice 25: x alpha value of -0.0023501324
indice 25: x beta value of 0.006538419
indice 26: y alpha value of 0.00051126635
indice 26: y beta value of 0.01640561
Final difference: 0.0019655562937259674

Total time: 988.0465100000147 s
```

```
Total iterations: 600
```

[8]:
```
# Optimization - Second Chromacity Quad

variables = {27: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {32: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 27 new current value: 4.86689453125001

y objectives:
indice 32: x disper value of 2.1372485e-07
Final difference: 4.567830827668408e-14

Total time: 65.24350899999263 s
Total iterations: 40
```

[9]:
```
# Optimization - Double Quadrupole Triplet

variables = {
    37: ["I", "current", lambda num:num],
    35: ["I2", "current", lambda num:num],
    33: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
    "I3": {"bounds": (0, 10), "start": 2},
}
objectives = {
    37: [
        {"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["y", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "envelope"], "goal": 2.0, "weight": 1},
        {"measure": ["y", "envelope"], "goal": 2.0, "weight": 1}
    ]
}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)

line[43].current = line[33].current
line[41].current = line[35].current
```

```
line[39].current = line[37].current
```

```
x variables:
indice 37 new current value: 1.0945677869001273
indice 35 new current value: 3.286802031305874
indice 33 new current value: 2.508548392246833

y objectives:
indice 37: x alpha value of -0.001548032
indice 37: y alpha value of -0.0015288863
indice 37: x envelope value of tensor(1.8348)
indice 37: y envelope value of tensor(1.9109)
Final difference: 0.00881356094032526

Total time: 541.6606401999888 s
Total iterations: 304
```

[10]:
```
# Optimization - Third Chromacity Quad

variables = {50: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {55: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 50 new current value: 4.750976562500008

y objectives:
indice 55: x disper value of -1.0166735e-06
Final difference: 2.2552006919904222e-13

Total time: 66.28704329999164 s
Total iterations: 40
```

[11]:
```
# Optimization - Quadrupole Doublet and Interaction Point (z = 7.11 m, end of␣
 ↪element index = 59)

variables = {
    56: ["I", "current", lambda num:num],
    58: ["I2", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
```

```
    "I2": {"bounds": (0, 10), "start": 2},
}

objectives = {
    59: [
        {"measure": ["x", "envelope"], "goal": 0.0, "weight": 1},
        {"measure": ["y", "envelope"], "goal": 0.0, "weight": 1}
    ]
}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 56 new current value: 3.123303213237886
indice 58 new current value: 3.322208105414836

y objectives:
indice 59: x envelope value of tensor(0.0158)
indice 59: y envelope value of tensor(0.0596)
Final difference: 0.0018997802399098873

Total time: 127.7826280000154 s
Total iterations: 71
```

[12]:
```
# Optimization - Quadrupole Doublet

variables = {
    61: ["I", "current", lambda num:num],
    63: ["I2", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
}

objectives = {
    68: [{"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "beta"], "goal": 0.1, "weight": 0.5}],
    69: [{"measure": ["y", "alpha"], "goal": 0, "weight": 1},
         {"measure": ["y", "beta"], "goal": 0.1, "weight": 0.5}]
}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 61 new current value: 5.179533592984713
indice 63 new current value: 4.041505095677061

y objectives:
indice 68: x alpha value of -0.00031931465
indice 68: x beta value of 0.03047347
indice 69: y alpha value of 0.0011084495
indice 69: y beta value of 0.019083465
Final difference: 0.0014227638021111488

Total time: 167.69443659999524 s
Total iterations: 91
```

[13]:
```
# Optimization - Fourth Chromacity Quad

variables = {70: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {75: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 70 new current value: 4.6065429687500075

y objectives:
indice 75: x disper value of 1.8583168e-07
Final difference: 3.453341467494521e-14

Total time: 64.93050989997573 s
Total iterations: 40
```

[14]:
```
# Optimization - Quadrupole Triplet

variables = {
    76: ["I", "current", lambda num:num],
    78: ["I2", "current", lambda num:num],
    80: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
    "I3": {"bounds": (0, 10), "start": 2},
}
```

```
objectives = {
    85: [{"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "beta"], "goal": 0.1, "weight": 0.5}],
    86: [{"measure": ["y", "alpha"], "goal": 0, "weight": 1},
         {"measure": ["y", "beta"], "goal": 0.1, "weight": 0.5}]
}

result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 76 new current value: 3.967454525510907
indice 78 new current value: 4.696816042181361
indice 80 new current value: 0.7932221734447746

y objectives:
indice 85: x alpha value of 0.00020130981
indice 85: x beta value of 0.081359625
indice 86: y alpha value of 0.000100227466
indice 86: y beta value of 0.08712892
Final difference: 6.415365351131186e-05

Total time: 768.3680936000019 s
Total iterations: 411
```

[15]:
```
# Optimization - Fifth Chromacity Quad

variables = {87: ["I", "current", lambda num:num]}
startPoint = {"I": {"bounds": (0,10), "start": 1}}
objectives = {92: [{"measure": ["x", "dispersion"], "goal": 0, "weight": 1}]}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
 ↪plotBeam=False, printResults=True, plotProgress=False)
```

```
x variables:
indice 87 new current value: 3.858984375000006

y objectives:
indice 92: x disper value of -3.1889954e-07
Final difference: 9.27157995018503e-14

Total time: 71.09410260000732 s
Total iterations: 40
```

[16]:
```
# Optimization - Quadrupole Triplet and MkIII undulator start (z = 12.389 m ,␣
↪end of element index = 117)

variables = {
    93: ["I", "current", lambda num:num],
    95: ["I2", "current", lambda num:num],
    97: ["I3", "current", lambda num:num],
}
startPoint = {
    "I": {"bounds": (0,10), "start": 2},
    "I2": {"bounds": (0, 10), "start": 2},
    "I3": {"bounds": (0, 10), "start": 2},
}
objectives = {
    117: [
        {"measure": ["x", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["y", "alpha"], "goal": 0, "weight": 1},
        {"measure": ["x", "beta"], "goal": beta_ym, "weight": 1},
        {"measure": ["y", "beta"], "goal": beta_ym, "weight": 1}
    ]
}
result = opti.calc("Nelder-Mead", variables, startPoint, objectives,␣
↪plotBeam=False, printResults=True, plotProgress=False)


#indice 93 new current value: 2.1275944045328767
#indice 95 new current value: 1.543397467601471
#indice 97 new current value: 0.3798657737076452
```

```
x variables:
indice 93 new current value: 1.34696857028191
indice 95 new current value: 3.517837310897537
indice 97 new current value: 2.2855047885846718

y objectives:
indice 117: x alpha value of 0.010050691
indice 117: y alpha value of 0.0011485886
indice 117: x beta value of 0.088548444
indice 117: y beta value of 0.22216971
Final difference: 0.005996518302708864

Total time: 1228.2800755999924 s
Total iterations: 549
```

[17]:
```
# Display Optimized Beamline

%matplotlib ipympl
acceptance = {"shape":'circle', "radius":1, "origin":[0,0]}
schem.plotBeamPositionTransform(beam_dist, line, 0.01, plot=True,␣
 ↪showIndice=False,
                                defineLim=False, matchScaling=False,␣
 ↪shape=acceptance, scatter=True)
```

```
Simulating Beamline:   0%|
| 0/1550 [00:00<?]

Simulating Beamline:   2%|
| 36/1550 [00:00<00:04]

Simulating Beamline:   5%|
| 72/1550 [00:00<00:04]

Simulating Beamline:   7%|
| 107/1550 [00:00<00:04]

Simulating Beamline:   9%|
| 143/1550 [00:00<00:04]

Simulating Beamline:  11%|
| 177/1550 [00:00<00:04]

Simulating Beamline:  13%|
| 207/1550 [00:00<00:07]

Simulating Beamline:  15%|
| 238/1550 [00:01<00:07]

Simulating Beamline:  17%|
| 260/1550 [00:01<00:07]

Simulating Beamline:  19%|
| 295/1550 [00:01<00:05]

Simulating Beamline:  21%|
| 327/1550 [00:01<00:05]

Simulating Beamline:  23%|
| 354/1550 [00:01<00:06]

Simulating Beamline:  25%|
| 391/1550 [00:01<00:05]

Simulating Beamline:  27%|
| 418/1550 [00:01<00:06]

Simulating Beamline:  29%|
| 446/1550 [00:02<00:05]
```

```
 Simulating Beamline:  30%|
| 472/1550 [00:02<00:05]

 Simulating Beamline:  32%|
| 498/1550 [00:02<00:04]

 Simulating Beamline:  34%|
| 527/1550 [00:02<00:04]

 Simulating Beamline:  36%|
| 556/1550 [00:02<00:03]

 Simulating Beamline:  38%|
| 583/1550 [00:02<00:05]

 Simulating Beamline:  40%|
| 619/1550 [00:02<00:04]

 Simulating Beamline:  42%|
| 645/1550 [00:03<00:05]

 Simulating Beamline:  44%|
| 679/1550 [00:03<00:04]

 Simulating Beamline:  46%|
| 714/1550 [00:03<00:03]

 Simulating Beamline:  48%|
| 751/1550 [00:03<00:02]

 Simulating Beamline:  51%|
| 788/1550 [00:03<00:02]

 Simulating Beamline:  53%|
| 821/1550 [00:03<00:03]

 Simulating Beamline:  55%|
| 856/1550 [00:03<00:02]

 Simulating Beamline:  57%|
| 885/1550 [00:04<00:03]

 Simulating Beamline:  59%|
| 909/1550 [00:04<00:04]

 Simulating Beamline:  61%|
| 947/1550 [00:04<00:03]

 Simulating Beamline:  63%|
| 972/1550 [00:04<00:03]

 Simulating Beamline:  65%|
| 1010/1550 [00:04<00:02]

 Simulating Beamline:  67%|
| 1036/1550 [00:04<00:02]
```

```
 Simulating Beamline:
69%|
| 1070/1550 [00:04<00:02]

 Simulating Beamline:
71%|
| 1097/1550 [00:05<00:01]

 Simulating Beamline:
73%|
| 1127/1550 [00:05<00:01]

 Simulating Beamline:
75%|
| 1155/1550 [00:05<00:01]

 Simulating Beamline:
76%|
| 1179/1550 [00:05<00:01]

 Simulating Beamline:
78%|
| 1202/1550 [00:05<00:01]

 Simulating Beamline:
79%|
| 1225/1550 [00:05<00:01]

 Simulating Beamline:
80%|
| 1247/1550 [00:05<00:01]

 Simulating Beamline:
83%|
| 1281/1550 [00:05<00:01]

 Simulating Beamline:
85%|
| 1310/1550 [00:06<00:00]

 Simulating Beamline:
87%|
| 1345/1550 [00:06<00:00]

 Simulating Beamline:
89%|
| 1379/1550 [00:06<00:00]

 Simulating Beamline:
91%|
| 1409/1550 [00:06<00:00]

 Simulating Beamline:
93%|
```

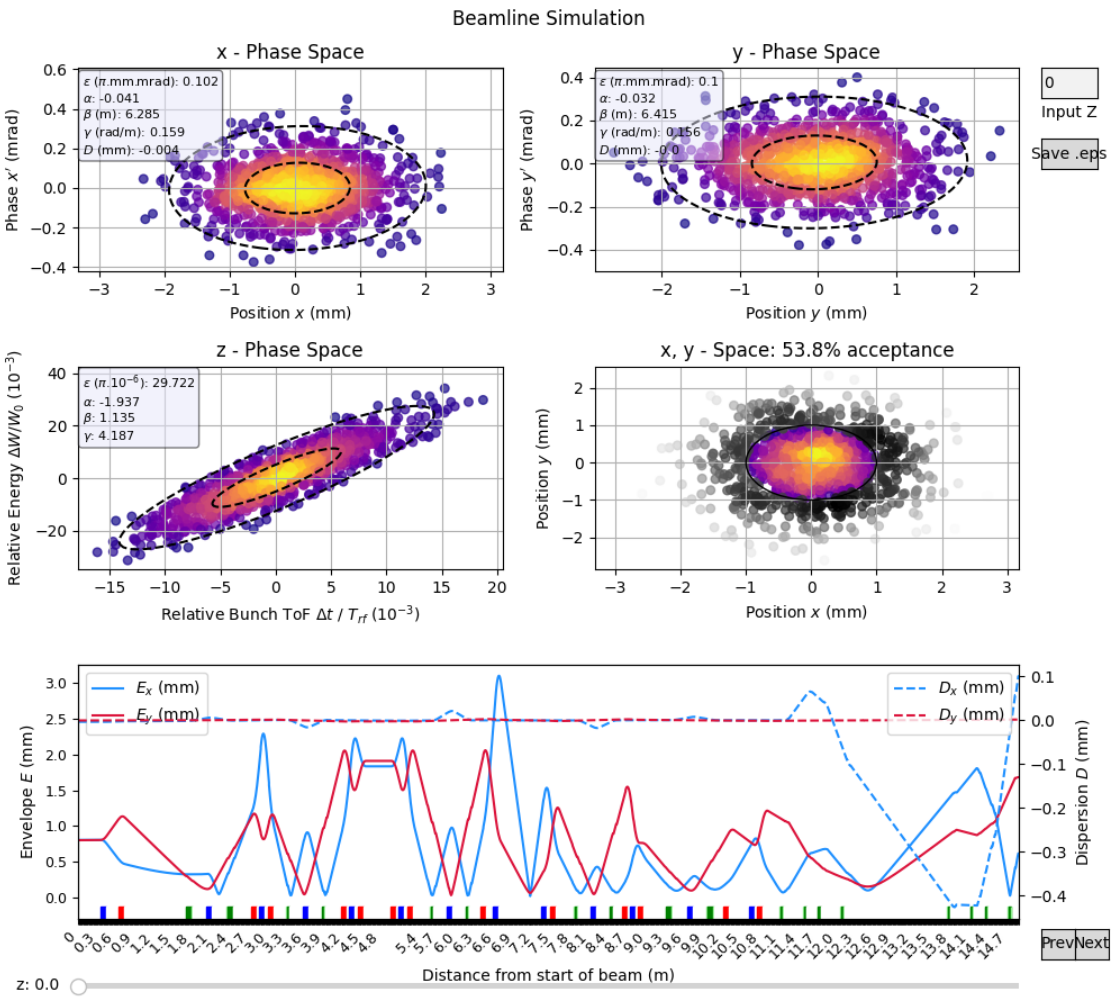
```
| 1439/1550 [00:06<00:00]

 Simulating Beamline:
95%|
| 1466/1550 [00:06<00:00]

 Simulating Beamline:
96%|
| 1490/1550 [00:06<00:00]

 Simulating Beamline:  98%|
                | 1512/1550 [00:06<00:00]

 Simulating Beamline:  98%|
                | 1514/1550 [00:06<00:00]
```



[17]:

```
                                $\epsilon$ ($\pi$.mm.mrad)  \
x  [0.10247279703617096, 0.10247278958559036, 0.1…
y  [0.09955935180187225, 0.09955935180187225, 0.0…
z  [29.721843719482422, 29.721843719482422, 29.72…

                                                $\alpha$  \
x  [-0.04066537693142891, -0.04225922375917435, -…
y  [-0.0319146104156971, -0.03347506374120712, -0…
z  [-1.9374732971191406, -1.937410593032837, -1.9…

                                            $\beta$ (m)  \
x  [6.284526824951172, 6.285356044769287, 6.28621…
y  [6.414903163909912, 6.415556907653809, 6.41624…
z  [1.1354191303253174, 1.1353610754013062, 1.135…

                                        $\gamma$ (rad/m)  \
x  [0.1593841016292572, 0.1593841165304184, 0.159…
y  [0.15604576468467712, 0.15604576468467712, 0.1…
z  [4.186826229095459, 4.186826229095459, 4.18682…

                                                $D$ (mm)  \
x  [-0.0035088863223791122, -0.003513646079227328…
y  [-0.0004015238373540342, -0.000399866345105692…
z  [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, …

                                      $D^{\prime}$ (mrad)  \
x  [-0.00047595068463124335, -0.00047595068463124…
y  [0.00016574750770814717, 0.0001657475077081471…
z  [0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, …

                                            $\phi$ (deg)  \
x  [-89.61962890625, -89.60478210449219, -89.5899…
y  [-89.70785522460938, -89.693603515625, -89.679…
z  [-25.890302658081055, -25.88958740234375, -25…

                                        Envelope $E$ (mm)
x  [0.8024917705506974, 0.8025446825231363, 0.802…
y  [0.7991643140622898, 0.7992050345024342, 0.799…
z  [5.809195292606346, 5.809046776215623, 5.80889…
```

[ ]:

```
```